# LESSON 06

# 写一次 for 跑百遍

## for 循环

学完以后，写几行代码，程序就能替你重复做很多遍。

# 输出 1 到 100，怎么写？

repeat.cpp（只阅读）

```
5       cout << 1 << endl;
6       cout << 2 << endl;
7       cout << 3 << endl;
8       cout << 4 << endl;
9       cout << 5 << endl;
```

输出 1 到 5，写 5 行还能忍。

输出 1 到 100，就要写 100 行。

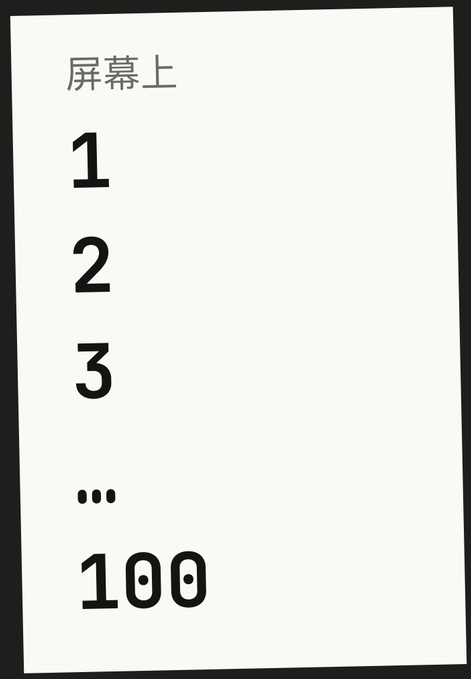
要是数到几由输入决定，写程序的时候根本不知道该写多少行。

这些行长得差不多，**只有数字在变**。

# 三行就够了

hundred.cpp

```
5       for (int i = 1; i <= 100; i++) {
6           cout << i << endl;
7       }
```




这就是 **for 循环**。

大括号里写要重复做的事，圆括号里写重复到什么时候。

`i` 是一个盒子，每轮装着不同的数。

# 圆括号里有三段

```
for (int i = 1; i <= 100; i++)
```

### 起点

准备盒子 i，先放进 1。这件事只做一次。

### 条件

每一轮开始前检查。成立就继续，不成立就结束。

### 更新

大括号运行完以后做，让 i 换成下一个数。

`i++` 是 `i = i + 1` 的简写，意思是把盒子里的数加 1。

三段之间用**两个分号**隔开，不是逗号。

# 一步步看：输入 3，第一轮

count.cpp

```
7       for (int i = 1; i <= n; i++) {
8           cout << i << endl;
9       }
```

大括号走完，回到第 7 行做更新：i++ 让 i 变成 2。
接下来不再做起点，直接查条件。

| | |
|---|---|
| i | 1 |
| 条件 | 真 |
| 屏幕 | 1 |
| i++ 后 | 2 |

# 接着走，走到停下来

| | | |
|---|---|---|
| i 是 1 | 1 <= 3 成立 | 输出 1，i 变成 2 |
| i 是 2 | 2 <= 3 成立 | 输出 2，i 变成 3 |
| i 是 3 | 3 <= 3 成立 | 输出 3，i 变成 4 |
| i 是 4 | 4 <= 3 不成立 | 循环结束 |

i 变成 4 以后，条件才不成立，所以 4 没有输出。

循环要靠更新让条件有机会变假，否则停不下来。

# 小测：输入 3，条件一共检查几次?

1 2 次

2 3 次

3 4 次

4 5 次

答对了！i 是 1、2、3、4 时各检查一次，最后一次不成立，循环结束。

# 猜一猜：输入 0 会怎样?

count.cpp

```
7       for (int i = 1; i <= n; i++) {
8           cout << i << endl;
9       }
```




什么也没有。for 先查条件，再决定要不要进大括号。
条件一开始就不成立，大括号**一次也不运行**。

# 动手写一写

先自己写，写不出来再翻回去看一眼。

## 输出 1 到 n

读入整数 n，是 0 到 100。每个数占一行。写完运行三次，每次输入一个数。

输入 3

```
1
2
3
```

输入 1

```
1
```

输入 0

```
什么也没有
```

Dev-C++: 打开 starter/02_count.cpp

# 猜一猜：少一个等号

less.cpp

```
7       for (int i = 1; i < n; i++) {
8           cout << i << endl;
9       }
```

屏幕上

1

2

<= 包含 n，< 不包含 n。

写完循环，先看**最后一个数**对不对。

# 小测：想输出 1 到 10，圆括号里写什么？

1 `int i = 1; i < 10; i++`

2 `int i = 1; i <= 10; i++`

3 `int i = 0; i <= 10; i++`

4 `int i = 1; i <= 9; i++`

答对了！从 1 开始，10 <= 10 也成立，所以 10 也会输出。

# 倒着数：三段都要换

down.cpp

```
7       for (int i = n; i >= 1; i--) {
8           cout << i << endl;
9       }
```

输入 3，屏幕上

3
2
1

**i = n**

从 n 开始

**i >= 1**

还没到 1 就继续

**i--**

每轮减 1，就是 i = i - 1

# 动手改一改

想清楚这一句写在大括号里面，还是外面。

## 数完喊 Go!

n 是 0 到 100 的整数。倒着数到 1，最后输出一次 Go!。

输入 3

```
3
2
1
Go!
```

输入 0

```
Go!
```

Dev-C++: 打开 starter/04_liftoff.cpp

# 动手写一写

起点不一定是 1。起点和终点都可以是变量。

## 从 a 数到 b

一行读入 a 和 b，都是 1 到 100 的整数，a 不大于 b，用空格隔开。从 a 输出到 b，每个数占一行。

输入 4 6

```
4
5
6
```

输入 1 1

```
1
```

Dev-C++：打开 starter/05_range.cpp

# 跳着数：改更新那一段

even.cpp

```
7       for (int i = 2; i <= n; i += 2) {
8           cout << i << endl;
9       }
```

输入 7，屏幕上

2

4

6

**i += 2** 是 i = i + 2 的简写，每轮加 2。

输入 7 时，i 依次是 2、4、6、8。

8 <= 7 不成立，所以 8 没有输出。

# 小测：这个循环输出哪些数？

```
for (int i = 1; i <= 10; i += 3)
```

1. 1 4 7
2. 1 4 7 10
3. 3 6 9
4. 1 2 3

答对了！i 依次是 1、4、7、10，变成 13 时才不成立。

# 动手改一改

在上一个程序上
改两个地方。

## 5 个 5 个地数

读入 n，是 0 到 1000 的整数。从 5 开始，每次加 5，不超过 n。每个数占一行。

输入 12

```
5
10
```

输入 15

```
5
10
15
```

输入 4

什么也没有

Dev-C++: 打开 starter/07_fives.cpp

# 大括号里可以拿 i 来算

table.cpp

```
7       for (int i = 1; i <= 9; i++) {
8           cout << n << " x " << i << " = " << n * i << endl;
9       }
```

输入 3，屏幕上

```
3 x 1 = 3
3 x 2 = 6
…
3 x 9 = 27
```

乘数每轮都在变，i 正好每轮都不一样。

把 i 放进算式，n 的乘法表就出来了。

n 是 0 到 100 的整数。

# 猜一猜：这次没有 endl

stars.cpp

```
7       for (int i = 1; i <= n; i++) {
8           cout << "*";
9       }
10      cout << endl;
```

屏幕上

****

endl 写在大括号外面，循环结束后只运行一次，
把这一行结束。放进大括号，每颗星就各占一行。

# 动手改一改

一个循环做完，下面还可以再写一个循环。

## 再来一行 #

n 是 1 到 50 的整数。第一行 n 个 *，第二行 n 个 #。

| 输入 3 | 输入 1 |
| --- | --- |
| *** | * |
| ### | # |

Dev-C++: 打开 starter/10_two_rows.cpp

# 读报错：i 怎么不见了？

count.cpp（只阅读）

```
5       for (int i = 1; i <= 3; i++); {
6           cout << i << endl;
7       }
```

```
6:17: error: 'i' was not declared in this scope
```

**not declared** 的意思是：这里没有这个盒子。

第 6 行没写错，错在第 5 行末尾多了一个分号。

分号让 for 到这里就结束了，下面的大括号不再属于它。

for 里准备的 i 只能在循环里用，所以第 6 行找不到 i。删掉分号就好。

Dev-C++ 显示的文字可能不一样，还是先找行号和 error。

# 找错：哪一行有问题？

five.cpp（只阅读）

```
1       for (int i = 1; i <= 5; i--) {
2           cout << i << endl;
3       }
4       cout << "Done" << endl;
```

这段程序想输出 1 到 5。
运行后却一直往下刷：
1、0、-1、-2……
很久也停不下来。
点你认为有问题的那一行。

点代码里的一行

遇到这种情况，关掉黑色的运行窗口，回来改程序。

# 自己挑战

# 小青蛙跳格子

一行读入三个整数，用空格隔开：起点 a（0 到 100）、每次跳的格数 d（1 到 10）、跳的次数 n（0 到 100）。

规则

**青蛙从第 a 格出发**

**每次往前跳 d 格**

**每跳完一次，输出它站在第几格**

输入 2 3 4，输出

**5**

**8**

**11**

**14**

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　循环要转几圈？这个数是三个输入里的哪一个？

**提示 2**　青蛙的位置每跳一次怎么变？想想第 2 课的 `age = age + 1`。

**提示 3**　这题 `i` 只负责数次数，位置放在另一个盒子里。
每轮先改位置，再输出。

# # 造一个你的小程序

读入一个整数，让程序重复做一件事。主题你自己定。

## 先做到

用 `for` 重复输出，重复几次由输入决定。

## 选你喜欢的

整点钟声、生日倒数、一排图案、乘法表，都可以。

## 还想更厉害

再读入起点和步长，或者连写两个循环，做出更长的输出。

没有标准答案，能运行、你喜欢就好。

# # 和同桌换着玩

## 1

### 换着运行

运行同桌的程序，试试输入 `0` 和一个大一点的数

## 2

### 说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

## 3

### 提一个想法

问问同桌：还可以多重复点什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

# 说得清吗?

**Q1** **起点、条件、更新，各在什么时候运行?**

起点只运行一次，条件每轮开始前查，更新在大括号运行完以后做。

**Q2** **输入 3，大括号运行 3 次，为什么条件查了 4 次?**

`i` 变成 `4` 时还要再查一次，这次不成立，循环才结束。

**Q3** **一句话写在大括号里和写在外面，有什么不同?**

写在里面每轮都运行，写在外面等循环结束后运行一次。

# 速查卡

```
for (int i = 1; i <= n; i++) { … }
```

| | |
|---|---|
| `int i = 1` | 起点：只运行一次 |
| `i <= n` | 条件：每轮先查，成立才进大括号 |
| `i++` | 更新：大括号运行完以后加 1 |
| `i--`　`i += 2` | 每轮减 1，每轮加 2 |
| `<=`　`<` | <= 包含 n，< 不包含 n |

# 这些后面再学

### while

事先不知道要重复几次时用它

### 循环里放 if

从一串数里挑出想要的

### 循环里放循环

一行重复，一整行也重复，画出图形

今天学会 for，知道起点、条件、更新各管什么，重复的事就可以交给程序了。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

**github.com/lixuanan2**

回到课程首页

下一课“while循环”还在准备中